

Feasibility of Homomorphic Inference for a Genomic Foundation Model

Christos Galanopoulos, Kimon Antonios Provas, and Ilias Georgakopoulos-Soares

Abstract—Human genomic sequences can identify individuals, cannot be replaced after disclosure, and are the inputs that genomic foundation models are designed to interpret. We assess whether a compute provider can execute a released genomic foundation model without receiving query-derived genomic values in plaintext and whether correctness, memory, or cost prevents complete encrypted inference. We first reproduce the released model on three genomic task families and freeze an independently validated numerical reference. We then implement a client-assisted approximate homomorphic encryption protocol: the provider evaluates linear algebra on ciphertexts, while the key-holding data owner evaluates exact normalization, causal softmax, and activation functions at fixed boundaries. A non-interactive configuration completes one released-weight block but exceeds the tested accelerator-memory envelope when configured for composition. The client-assisted configuration executes all released transformer blocks and the task head for one held-out genomic-signal input at its full prompt length. It matches the frozen final label, peaks at 9,839 mebibytes of accelerator memory, and completes in 6,683 seconds on one accelerator. These results establish arithmetic feasibility for a complete classifier, while repeatability, network transport, and private token-index lookup remain unresolved. The biomedical significance is that, under the stated threat model, a served genomic model can process an encoded sequence without exposing plaintext query-derived activations to the compute provider.

Index Terms—genomic privacy, homomorphic encryption, privacy-preserving inference, transformers.

I. INTRODUCTION

Human genomic sequence carries a persistent confidentiality risk. It can identify its contributor and reveal information about relatives; re-identification has been demonstrated after conventional identifiers were removed [1], [2]. A disclosed genome cannot be replaced like a password. European data-protection law accordingly treats genetic data as a special category [3]. Yet genomic foundation models derive their value from this same sequence: DNAGPT supports classification, regression, and generation through a common transformer backbone [4]. Homomorphic encryption offers access to a served model without sending the provider plaintext genomic values.

DNAGPT’s public release makes this experiment auditable and permits local execution. We use it as a reproducible surrogate for a genomic model offered only as a service by policy or contract, accessed by a CPU-equipped data owner without accelerator hardware. We ask whether its released computation can execute faithfully within the cryptographic and memory budget, and whether the resource cost supports a target use. These are separate feasibility and practicality

verdicts: a correct but slow encrypted path can resolve the former while leaving the latter open.

The technical difficulty is concentrated in transformer nonlinearities. CKKS supports approximate packed arithmetic for dense projections, rotations, and accumulations [5], whereas non-interactive LayerNorm, softmax, and GELU require polynomial approximations that increase depth and resident cryptographic material. Moving exact functions to the key holder resets the depth budget but introduces interaction and a different threat model. THOR demonstrates non-interactive homomorphic transformer inference; Saphire uses the server-linear/client-nonlinear division in convolutional networks [6], [7]. We study the correctness, memory, and executed work of the latter established pattern on a released genomic transformer.

We first evaluate genomic-signal recognition, promoter and splice-site classification, and mRNA abundance regression, freezing outputs before encrypted work. An independent NumPy reference is asserted against upstream PyTorch at 2×10^{-5} for every released block and the task head. We then build non-interactive and client-assisted CKKS designs, with fixed, non-adaptive boundaries for exact nonlinearities in the latter. Instrumented evaluations record correctness and memory; runtime assertions preserve the cryptographic operation schedule during systems optimization.

The client-assisted protocol executes all 12 released blocks and the task head for one held-out input at the 103-token genomic-signal prompt. Its label matches the frozen reference, with head margin relative error 8.56×10^{-9} . The non-interactive design completed a released-weight block but exceeded the tested 80 GB envelope while loading its composition context and evaluation keys. The complete client-assisted run instead peaks at 9,839 MiB of process GPU memory and takes 6,683 s on one accelerator. Bounded caching also prevents encoded plaintexts from accumulating across stages. These results close arithmetic feasibility at the stated encrypted boundary while leaving deployment practicality a separate question.

The paper makes the following contributions:

- An independently verified plaintext reference across three genomic task families.
- A specified client-assisted CKKS protocol, including packing, depth resets, algorithms, and a semi-honest threat model.
- Complete released-weight inference for one held-out input, with label and intermediate verification under a predeclared tolerance.
- Fixed-circuit systems optimization with an asserted operation schedule, bounded host caching, and separate feasibility and practicality verdicts.

II. BACKGROUND

A. DNAGPT and the genomic task families

DNAGPT is an autoregressive transformer for sequence and numerical tasks over DNA [4]. The evaluated 0.1-billion-parameter configuration has 12 blocks, hidden width 768, 12 causal-attention heads of width 64, and feed-forward width 3,072. Its input combines 6-mer DNA tokens with task-format tokens.

Each block applies LayerNorm; query, key, and value projections; causal self-attention; an output projection and residual; a second LayerNorm; and an expanded MLP with GELU and another residual. Dense maps use fixed weights; LayerNorm, attention, and GELU introduce data-dependent products or nonlinear functions.

Released heads cover genomic-signal recognition and mRNA abundance regression; the genome-understanding benchmark uses a locally fine-tuned linear head (Section IV-E). These tasks follow convolutional signal recognition [8], abundance regression [9], and the encoder-based benchmark [10]. They establish model fidelity before encrypted evaluation of the signal-recognition path.

B. CKKS

CKKS supports approximate arithmetic on packed real or complex vectors [5]. Values occupy plaintext slots encrypted under the data owner’s key. Addition and multiplication act slotwise; rotations move values between slots. This single-instruction, multiple-data model expresses public matrix transforms through rotated ciphertexts, plaintext diagonals, and accumulation.

Query-key scoring and attention context instead multiply encrypted query-derived operands, requiring ciphertext-ciphertext multiplication and relinearization. Public weights and masks remain plaintext; query-derived operands remain encrypted at the server.

Rescaling controls fixed-point scale using levels from a finite modulus chain. Bootstrapping refreshes the budget without decryption; fresh client encryption resets it interactively. Their system costs make multiplicative depth an architectural resource.

C. Why the nonlinearities are the architecture

LayerNorm requires inverse square root, softmax requires exponentiation and normalization, and the MLP requires GELU. A non-interactive circuit approximates these functions polynomially over declared input ranges. Higher degree consumes additional depth, increasing the context and evaluation-key material needed across repeated blocks. Exact evaluation at declared client boundaries removes range calibration and resets depth, while requiring an online client that observes intermediate activations. This choice determines memory, interaction, and the threat model; Section V specifies both designs.

D. Software backend selection

OpenFHE provides CKKS context and security-parameter construction, encoding, rotations, polynomial evaluation, and approximate bootstrapping [11], establishing reference behavior for local operators and the non-interactive design. The evaluated GPU backend interoperates with those representations and supplies every measured server-side CUDA operation. Python, NumPy, and PyTorch construct and validate the plaintext reference outside the encrypted performance path.

We also considered Concrete ML and the Boolean/integer-oriented TFHE-rs ecosystem [12], [13]. Concrete ML documents a hybrid language-model protocol with client-side attention and activations and server-side linear layers [14]. This was a documentation-based assessment, not a local DNAGPT performance comparison. We retained the validated CKKS implementation; this engineering choice does not establish CKKS as the only viable encryption scheme.

III. RELATED WORK

A. Private transformer inference

Gazelle combines homomorphic linear layers with garbled-circuit nonlinearities [15]. Iron, BOLT, Nimbus, and BumbleBee adapt homomorphic encryption, secret sharing, and/or oblivious-transfer-based protocols to transformer attention, normalization, and activations [16]–[19]. They optimize joint secure computation; our server evaluates the linear path in CKKS and the data owner evaluates exact nonlinearities at explicit boundaries, changing both the security contract and the allocation of work.

The Sapphire preprint describes a closely related architecture: its client decrypts an intermediate, evaluates the nonlinearity, and returns fresh encryption [7]. It studies convolutional networks and adds randomized output permutation intended to hinder reconstruction from intermediates. We apply this established pattern to a released genomic transformer at its task-defined prompt, preserving released weights and exact nonlinearities. The contribution is the independently checked verification chain and measured arithmetic, boundary schedule, and memory behavior.

THOR demonstrates full non-interactive homomorphic transformer evaluation using diagonal-major organization and compact packing [6]. Thus, our observed memory wall is specific to the evaluated backend, parameters, packing, and circuit. Client assistance tests whether released computation can be preserved when the data owner is available for exact nonlinear evaluation.

B. Homomorphic encryption for genomics

The iDASH competition evaluated encrypted association studies, sequence search, and related genomic workflows [20]. Private genomic-query systems combine homomorphic encryption, hashing, and set intersection to test for variants without disclosing the genome or query [21]. Beyond these searches and prescribed statistics, DNAGPT inference composes dense projections, attention, normalization, and feed-forward layers while preserving the numerical behavior of a released foundation model.

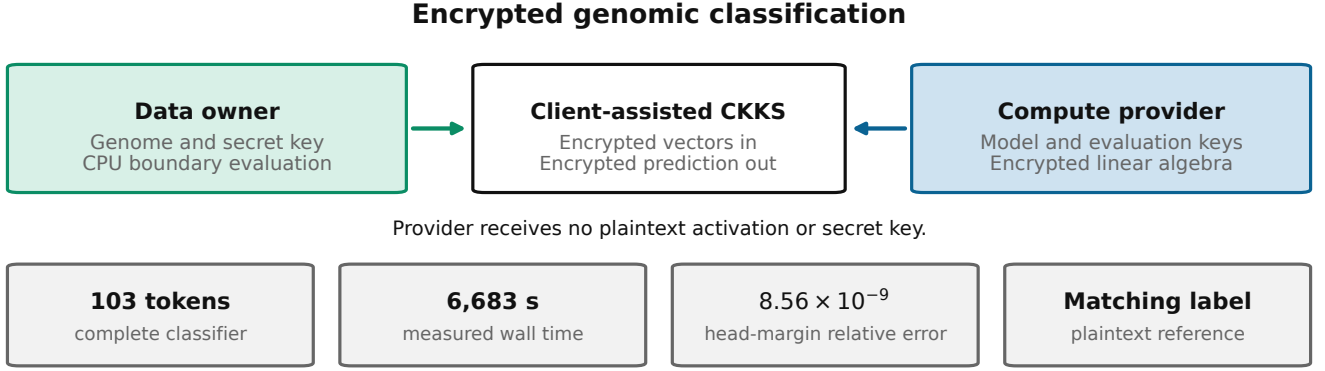


Fig. 1. Study overview and measured complete-classifier result. The data owner retains the secret key and evaluates exact nonlinearities; the compute provider evaluates encrypted linear algebra. The reported execution begins at encrypted embedded vectors and covers one genomic-signal input.

C. GPU-accelerated homomorphic encryption

The evaluated GPU backend supplies number-theoretic transforms, key switching, rotations, and ciphertext arithmetic interoperable with the host runtime [11], [22]. Its missing ciphertext serialization prevented process-sharded evaluation, leaving thread-level concurrency and in-process device placement. Multi-GPU transformer work motivates further placement and communication-computation overlap [23]; adopting those schedules requires direct evaluation of serialization and transport, which this implementation does not exercise.

D. Homomorphic matrix primitives

Matrix layout determines rotations, plaintext encodings, and reductions. THOR specializes it for transformers [6]; HE-BLAS supplies homomorphic matrix-vector, vector-vector, and matrix-matrix reductions implemented with BLAS [24]. These motivate less repeated packing, fewer diagonal copies, and shape-matched reductions (Section VI). Because packing, keys, levels, and the operation schedule are coupled, candidate primitives must be revalidated against the frozen reference and schedule before performance comparison.

IV. STUDY DESIGN AND THREAT MODEL

A. Deployment roles and privacy objective

Figure 1 summarizes the deployment and complete-classifier evaluation. The deployment has two parties but one cryptographic confidentiality objective. The data owner holds a human genomic sequence, ordinary CPU resources, and no GPU. It will not disclose the sequence to the compute provider. The model owner operates the accelerator infrastructure and serves the DNAGPT model rather than distributing it to clients. Keeping the weights server-side is an operational preference of this deployment, not a model-confidentiality guarantee supplied by the protocol.

The released DNAGPT artifact makes the experiment reproducible; it does not itself require this service arrangement. A client may run the public release locally. We use it as an auditable surrogate for a genomic model whose owner limits access to a service by policy or contract, and the deployment conclusions apply under that premise.

The protected asset is therefore the genome. The data owner encrypts its embedded numeric vectors and retains the secret key; the compute provider evaluates the served model on those ciphertexts. The protocol is relevant only where these roles are fixed. If a client is allowed to receive and run the model, local plaintext inference is the simpler architecture.

B. Local inference as a comparison

Where model distribution is permitted, the client can avoid all cryptographic work by running the ordinary forward pass. Client assistance cannot provide a computational advantage over that option. Its purpose is to make a service-only model available without sending the genomic input in plaintext.

That comparison changes the deployment premise. A served model is available to the data owner only through the provider's interface; handing the client the weights creates a different service. Under the premise studied here, the available choices are to disclose the genome, decline the inference, or evaluate through a privacy-preserving protocol. Client-assisted CKKS addresses the last choice. The reason to use it is access to a served model without genomic disclosure, not an attempt to beat plaintext inference.

The protocol assigns encrypted dense linear algebra to the party with accelerator hardware while the key holder evaluates exact nonlinearities on CPU. This allocation protects the genomic input without requiring the data owner to operate a GPU.

This allocation becomes more server-heavy as model width grows. At fixed sequence structure, the client's boundary work scales with the number of activation values, whereas the server's dense transforms scale quadratically with hidden width. The client fraction therefore decreases as $O(1/D)$ with hidden width D under this fixed sequence structure. This is an asymptotic property of the work allocation, not a measurement of a wider DNAGPT model.

C. Adversary model

We consider a semi-honest compute provider under 128-bit classical CKKS parameters. It follows the prescribed circuit but may inspect everything it legitimately receives: public and

TABLE I
RESOURCE AND TRUST ALLOCATION UNDER LOCAL AND SERVED-MODEL DEPLOYMENT.

	Client-local inference	Client-assisted CKKS
Client must hold the model	yes	no
Server observes the genome	not applicable	no
Client requires a GPU	deployment-dependent	no
Server evaluates model linear algebra	no	on ciphertexts
Client evaluates model nonlinearities	as part of local model	at declared boundaries

evaluation keys, model parameters, ciphertexts, sequence length, message sizes, operation order, boundary schedule, and timing. It does not alter ciphertexts to probe the client or deviate from the declared boundary sequence.

Under this adversary, the compute provider receives no plaintext genomic vector, plaintext activation, partial decryption, or secret-key material. At a boundary it supplies a ciphertext and receives a fresh ciphertext under the data owner’s public key. The guarantee concerns the provider’s view of query-derived values; public model structure and plaintext weights remain available to the provider as inputs to ciphertext–plaintext operations.

Invariant 1 (Key custody). *The data owner generates and retains the secret key. No protocol message contains the key or a partial decryption, and the compute provider has no decryption operation.*

Invariant 2 (No plaintext at the server). *Every query-derived value held by the compute provider is a ciphertext under the data owner’s key. Client-evaluated nonlinearities return through fresh encryption rather than as decoded values.*

Invariant 3 (Non-adaptive boundaries). *The circuit and sequence length determine every boundary before execution; decrypted content does not select the next operation. The implementation asserts the complete schedule and boundary counts for each accepted run and aborts on deviation. Transcript equality across multiple private inputs of equal length has not been measured as a separate leakage experiment.*

D. What is not claimed

Model confidentiality is not claimed. The data owner observes full intermediate activations at every nonlinearity. Those observations decouple the network into shallow segments with known nonlinearities, turning global model inversion into inexpensive per-segment regression. Keeping weights on the server may deter casual copying, but it does not prevent extraction by a determined client. This concession does not change the genomic-data guarantee, which protects the data owner from the compute provider.

Malicious servers, authenticated transport, production key custody, traffic analysis, timing and physical side channels, and compromised clients are outside the evaluated boundary. Encrypted scope begins after token-index lookup, so private lookup is also unresolved. The data owner sees its own intermediate activations and final output; privacy from the key holder is not an objective of this protocol.

E. Plaintext reference and experimental design

An encrypted execution is useful only if the computation it reproduces is itself meaningful and independently checked. Task metrics answer the first question: they establish that the released model and locally trained heads retain the genomic capabilities for which the model is being evaluated. Per-example predictions answer the second. They are recorded before any encrypted work and become the frozen plaintext reference against which encrypted outputs are accepted.

The plaintext harness imports the upstream model implementation and preserves its classification and regression forward paths. It adds batching, metric computation, and result recording without changing model arithmetic. Evaluation uses the canonical test split for each task, and each run records both its configuration and the per-example outputs needed downstream. Agreement with a published aggregate metric is evidence that the intended model behavior survived local reproduction; the recorded output for the selected encrypted input is the stricter numerical contract used by the cryptographic evaluation.

The reference is not trusted merely because it is written in NumPy. Released checkpoint weights are evaluated through the upstream PyTorch block and, independently, through a float64 NumPy re-derivation. For every one of the 12 transformer blocks, the two outputs are asserted equal at relative and absolute tolerance 2×10^{-5} ; the classifier head is checked by the same procedure. Generation aborts on any divergence. Only after this upstream-to-reference comparison passes are the weights and reference output supplied to the encrypted circuit. This creates two independent checks: the plaintext re-derivation must reproduce the released model, and the encrypted computation must reproduce that validated re-derivation.

F. Task-derived sequence lengths

The encrypted anchor length follows from the genomic-signal task rather than from a convenient cryptographic setting. Its 600-base-pair window becomes 100 sequence tokens under the model’s 6-mer tokenization; three task-format special tokens bring the complete prompt to 103 tokens. The evaluated prompt is therefore the task input consumed by the released classifier, rather than a shortened surrogate chosen to fit the encrypted implementation.

The same arithmetic determines the other classification prompts. Core-promoter detection maps a 70-base-pair window to 12 sequence tokens plus one special token, or 13 total. The 300-base-pair promoter task uses 50 sequence tokens plus one special, or 51; the 400-base-pair splice task uses

TABLE II

NOTATION. ALL VALUES ARE THOSE OF THE EVALUATED CONFIGURATION.

Symbol	Value	Meaning
D	768	hidden width of the released model
H	12	attention heads, of $D/H = 64$ dimensions each
D_{mlp}	3072	MLP expansion width, $4D$
T	103	tokenized prompt length for the genomic-signal task
B	8	tokens packed per ciphertext (token lanes)
G	13	ciphertext groups, $\lceil T/B \rceil$
W	1024	feature width padded to a power of two
C	4	activation copies per ciphertext, D_{mlp}/D
N	65536	polynomial ring degree
$N/2$	32768	usable plaintext slots per ciphertext, $C \cdot W \cdot B$
L	13	multiplicative depth
λ	128	classical security level, in bits

67 sequence tokens plus one special, or 68. The 103-token genomic-signal prompt is the longest of these encrypted-scope tasks and consequently provides the anchor for the packing and sequence-length analysis in Sections V and VII.

V. CLIENT-ASSISTED CKKS PROTOCOL

Table II defines the evaluated configuration. Figure 2 locates the server computation and the client calls within a transformer block. The protocol retains the released weights and exact nonlinearities while bounding encrypted depth through fresh client encryption.

A. Client boundaries and normalization

CKKS carries affine maps, reductions, residual additions, and attention products. The inverse square root in LayerNorm, causal softmax, and GELU use declared client calls. At each call, the data owner decrypts values derived from its query, evaluates the function in double precision, and encrypts the result afresh under the same key. Algorithm 1 gives this operation.

Algorithm 1 The client boundary. The data owner is the only party holding sk , and every value it decrypts derives from its own query.

```

1: function CLIENTBOUNDARY( $c, f$ )
2:    $v \leftarrow \text{DECODE}(\text{DECRYPT}(sk, c))$ 
3:    $v' \leftarrow f(v)$   $\triangleright$  evaluated exactly, in double precision
4:   return  $\text{ENCRYPT}(pk, \text{ENCODE}(v'))$   $\triangleright$  fresh ciphertext at level 0
5: end function

```

Fresh encryption returns the activation to level 0. The configured depth of 13 therefore bounds the longest encrypted run between boundaries rather than the depth accumulated across all 12 transformer blocks. The ciphertext lineage remains at the server between these calls.

For LayerNorm, the server computes the mean and centered variance under encryption and adds ε before sending the variance term to the client. Only the inverse square root is

evaluated there; the server then applies the returned factor and public scale, as specified in Algorithm 2. No bias is omitted as an approximation: the released checkpoint contains no bias tensors.

Algorithm 2 LayerNorm. The server computes both statistics under encryption; only the inverse square root crosses the boundary. The released checkpoint carries no bias term.

```

1: function LAYERNORM( $x, \gamma$ )
2:    $\mu \leftarrow \text{REDUCESUM}(x) \odot (1/D)$ 
3:    $z \leftarrow x - \mu$ 
4:    $\sigma^2 \leftarrow \text{REDUCESUM}(z \odot z) \odot (1/D) + \varepsilon$ 
5:    $r \leftarrow \text{CLIENTBOUNDARY}(\sigma^2, t \mapsto t^{-1/2})$ 
6:   return  $(z \odot r) \odot \gamma$ 
7: end function

```

B. Packing and dense transforms

The slot layout in Fig. 3 assigns 32,768 slots as 4 activation copies by 1,024 padded features by 8 token lanes. Token lane is the innermost index: whole-lane rotations move features without mixing tokens, and smaller rotations align token lanes. The released width of 768 occupies the active feature region; padding supports the power-of-two transform and attention-score staging. Rows and columns outside the active width contribute zeros.

The four copies express the MLP expansion from width 768 to 3,072 as parallel chunks. The 103 tokens occupy 13 ciphertext groups. A 32×32 baby-step giant-step decomposition covers the 1,024 padded features. Algorithm 3 combines shared baby rotations with encoded weight diagonals, cloning each plaintext for use at the receiving ciphertext's level.

Algorithm 3 Packed matrix product, baby-step giant-step over the packed diagonals. $N_1 = N_2 = 32$ and $N_1 N_2 = W = 1024$.

```

1: function MATMUL( $\beta, M$ )
2:    $d \leftarrow \text{ENCODED}(\text{DIAGONALS}(M, \text{level}(\beta_0)))$   $\triangleright$  encoded once, cloned per use
3:    $y \leftarrow 0$ 
4:   for  $j \leftarrow 0$  to  $N_2 - 1$  do
5:      $u \leftarrow \sum_{i < N_1} \beta_i \odot \text{CLONE}(d_{N_1 j + i})$ 
6:      $y \leftarrow y + (j = 0 ? u : \text{ROT}(u, BN_1 j))$ 
7:   end for
8:   return  $y$ 
9: end function

```

Eight-token packing reduces the serial-equivalent dense-product count from 1,236 to 156, a structural reduction of $7.92\times$. Rotations, masking, encoding, and client work remain, so this operation-count ratio does not specify a latency speedup.

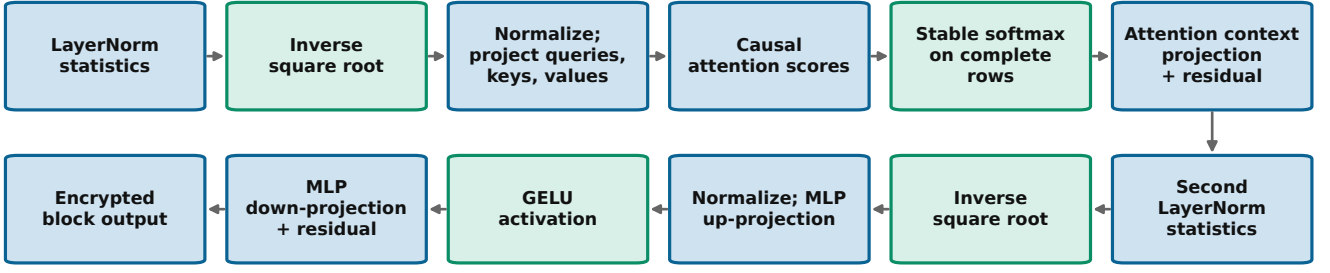
C. Block evaluation

Algorithm 4 follows the released block through normalization, query/key/value projection, causal attention, output projection, residuals, and the expanded MLP. The cached diagonals are flushed after the last use of each weight set; this changes storage lifetime without changing the arithmetic. Baby rotations are shared among transforms that consume the same input.

One transformer block: evaluation order and executor

Data owner: genome + secret key

Compute provider: model + evaluation keys



Client functions decrypt, evaluate in double precision, and re-encrypt; score collection and softmax emission are separate.

Fig. 2. Client-assisted block evaluation. The compute provider evaluates encrypted linear algebra and the data owner evaluates exact nonlinearities at declared boundaries. The provider has public weights, keys, and execution metadata, but receives no plaintext query-derived activation or secret key.

32,768 slots per ciphertext

4 copies × 1,024 features × 8 token lanes



768 active features; padding includes attention staging.

103 tokens → 13 groups



Final group: 7 live lanes + 1 padded lane.

Fig. 3. Slot layout and grouping for the measured prompt. The final ciphertext group carries seven live tokens and one padded lane; the groups produce the lower-triangular causal score tiles.

The protocol transcript is fixed by the circuit and sequence length. Each block makes 857 physical client boundary crossings, including 91 score-tile decrypts and 727 weight-tile encryptions. They carry 129,162 logical nonlinear instances. Softmax accounts for 128,544, or 99.5%, of them, concentrating the boundary work in attention. The implementation asserts the operation schedule, boundary counts, and remaining depth at run time and aborts on deviation.

D. Causal attention

Attention visits the 91 lower-triangular pairs of the 13 token groups. Shifts required by later query groups are computed once per key group and reused. The client collects all encrypted score tiles before stable softmax normalizes complete causal rows, then returns fresh encrypted weight tiles for context aggregation. This normalization is a global synchronization point.

Both query-key scoring and weight-value aggregation multiply two activation-derived operands. Of the block's 1,506 ciphertext-ciphertext multiplications, LayerNorm contributes 26 centered squares and 26 products with returned inverse

square roots. Attention contributes 727 query-key and 727 weight-value products, or 97% of the total.

E. Depth, parameters, and composition

The retained context has ring dimension 65,536, 32,768 usable slots, and a 128-bit classical security level. It uses 50-bit scaling, a 60-bit first modulus, 3 hybrid key-switching digits, and 80 rotation keys. Depth 13 is the smallest demonstrated passing depth for these parameters and packing; the block finishes with 6 levels remaining.

Between blocks, the data owner decrypts the full hidden state, restores its token-vector layout, and encrypts fresh level-0 inputs under the same context and key lineage. The preliminary two-block execution at two tokens reached relative error 8.48×10^{-11} without GPU-memory growth in the second block. Section VII reports the complete task-length composition.

F. Non-interactive comparison

The non-interactive design replaces every nonlinearity with a polynomial approximation and retains a single ciphertext lineage without intermediate decryption. It completes one released-weight block. When configured for composition, however, its context and evaluation keys exceed the tested 80 GB GPU-memory envelope before useful composition arithmetic begins.

Repeated polynomial evaluation requires a deeper modulus budget and a bootstrapping schedule whose associated material must remain resident. This memory limit applies to the tested library, parameters, packing, and hardware. The client-assisted design shortens encrypted segments by evaluating the exact functions at the client and refreshing through fresh encryption.

VI. SYSTEMS OPTIMIZATION AT FIXED CIRCUIT

The encrypted circuit remains fixed throughout this section: 156 dense products, 177,734 ciphertext-plaintext multiplications, 1,506 ciphertext-ciphertext multiplications, 8,173 rotations, and 857 client boundary crossings per block. Runtime assertions check these counts, the boundary schedule, and

Algorithm 4 Evaluating one transformer block. $G = 13$ token groups; $\odot$ is homomorphic multiplication; every operation outside a CLIENT call runs on ciphertexts.

Require: encrypted activations $c_0, \dots, c_{G-1}$; public weights W

Stage 1: normalization and projection

```

1: for  $g \leftarrow 0$  to  $G - 1$  do
2:    $n \leftarrow \text{LAYERNORM}(c_g, \gamma_1)$  ▷ Algorithm 2
3:    $\beta \leftarrow \text{BABYROTATIONS}(n)$  ▷  $N_1 - 1$  rotations, shared
4:    $Q_g \leftarrow \text{MATMUL}(\beta, W_q)$ ;  $K_g \leftarrow \text{MATMUL}(\beta, W_k)$ 
5:    $V_g \leftarrow \text{MATMUL}(\beta, W_v)$ 
6: end for
7:  $\text{FLUSHENCODEDWEIGHTS}$  ▷ bounds host memory to one stage

```

Stage 2: causal attention scores

```

8: for  $k \leftarrow 0$  to  $G - 1$  do
9:    $\tilde{K} \leftarrow \{\text{LANESHIFT}(K_k, \delta)\}$  for each  $\delta$  required by a later query group
10:  for  $q \leftarrow k$  to  $G - 1$  do
11:     $S \leftarrow 0$ 
12:    for  $\delta \in \text{ACTIVESHIFTS}(q, k)$  do
13:       $P \leftarrow Q_q \odot \tilde{K}_\delta$ 
14:      for  $h \leftarrow 0$  to  $H - 1$  do
15:         $S \leftarrow S + \text{REDUCESUM}(P \odot \text{mask}_h) \odot$ 
        isolate( $q, k, \delta, h$ )/ $\sqrt{d_h}$ 
16:      end for
17:    end for
18:     $\text{CLIENT.TAKESCORTILE}(S, q, k)$  ▷ decrypt only
19:  end for
20: end for
21:  $\text{CLIENT.SOFTMAX}$  ▷ stable softmax over complete causal rows

```

Stage 3: attention context

```

22: for  $k \leftarrow 0$  to  $G - 1$  do
23:    $\tilde{V} \leftarrow \{\text{LANESHIFT}(V_k, \delta)\}$ 
24:   for  $q \leftarrow k$  to  $G - 1$ ;  $\delta \in \text{ACTIVESHIFTS}(q, k)$  do
25:      $A \leftarrow \text{CLIENT.EMITWEIGHTTILE}(q, k, \delta)$  ▷ re-encrypt only
26:      $C_q \leftarrow C_q + A \odot \tilde{V}_\delta$ 
27:   end for
28: end for

```

Stages 4 and 5: projection, MLP, residuals

```

29: for  $g \leftarrow 0$  to  $G - 1$  do
30:    $R_g \leftarrow c_g + \text{MATMUL}(\text{BABYROTATIONS}(C_g), W_{\text{proj}})$ 
31: end for
32:  $\text{FLUSHENCODEDWEIGHTS}$ 
33: for  $g \leftarrow 0$  to  $G - 1$  do
34:    $\beta \leftarrow \text{BABYROTATIONS}(\text{LAYERNORM}(R_g, \gamma_2))$ 
35:    $h \leftarrow \sum_{c < 4} \text{MATMUL}(\beta, W_{\text{fc}}^c) \odot \text{copy}_c$ 
36:    $a \leftarrow \text{CLIENT.GELU}(h)$ 
37:    $c'_g \leftarrow R_g + \sum_{c < 4} \text{MATMUL}(\text{BABYROTATIONS}(a \odot \text{copy}_c), W_{\text{mlp}}^c)$ 
38: end for
39: return  $c'_0, \dots, c'_{G-1}$ 

```

remaining depth before the MLP, aborting on deviation. Packing, depth, reference, and acceptance criterion are unchanged. Optimizations alter host-data preparation, residency, and CPU placement without removing arithmetic or moving linear work to the client. The retained system completes the full classifier in one clean sample; the pre-optimization timing denominator lacks matching evidence quality, so we report the endpoint without a speedup factor.

A. Host-side encoding redundancy

The baseline rebuilt and encoded 1,024 diagonals for each of 156 dense products, yielding 159,744 encodings per block.

About 90% were byte-identical because the 12 weight matrices recur across 13 token groups. Each diagonal must be packed, scaled, and encoded at the receiving ciphertext’s level before GPU multiplication can begin. Caching this model-invariant representation therefore removes redundant host preparation while preserving encrypted products and accumulations.

B. Encode once, clone per use

The retained cache encodes each distinct weight diagonal into a pristine host template indexed by matrix and multiplicative level. Each multiplication receives a clone sharing encoded host data by reference but carrying fresh device state, loaded and evicted for that operation. This removes about 90% of the 159,744 encodings without persistent GPU plaintext objects.

Cloning follows the backend’s object semantics: its load path returns early for an already-loaded object, so direct reuse across levels can expose stale residue-number-system limbs. Fresh device state forces loading at the current level. An isolated GPU test verified clone-versus-fresh-encode equality, cross-level reuse, stable memory during repeated multiplication, and parallel-versus-serial encoding equality before application to a complete block.

C. Bounding host memory

An early cache retained templates across completed stages and failed allocation when the MLP constructed its templates above those left by projection and attention.

Two exact changes bound residency. Templates are encoded at their level of use, retaining only required limbs; the backend would otherwise drop a level-0 plaintext to that same level. The cache is then flushed after query/key/value projection and after attention projection, where the respective weights reach their final use. The group-major loop retains reuse within each stage; eviction between stages cannot create additional re-encoding.

The resident-set trace (Fig. 4) reaches 24.9 GiB during query/key/value projection and 12.1 GiB during attention after projection templates are flushed. The MLP’s eight matrices determine the 47.5 GiB block peak. Thus, the largest live stage determines peak host memory.

D. Placement and parallelism

The retained implementation binds threads to physical cores and uses NUMA-local allocation for encoding and GPU staging buffers. Decryption, nonlinear evaluation, and re-encryption share those host resources. Template construction uses dynamic scheduling across 32 cores: independent diagonals are encoded in parallel without changing encrypted multiplication order. Output equality was checked against serial construction. The individual timing effects of affinity and batching await sourced paired measurements.

E. Remaining optimization targets

About 17,400 mask plaintexts are still encoded afresh per block. Head, score-isolation, lane, and copy-activity masks are fixed by the layout; caching them remains unimplemented.

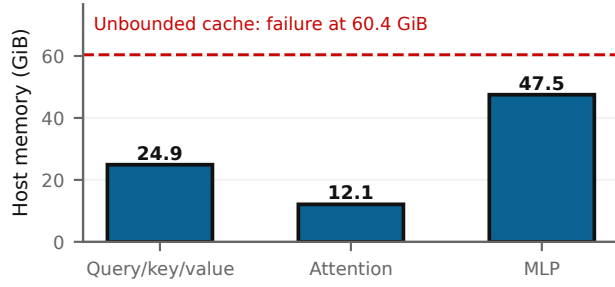
TABLE III

THE OPTIMIZATION CAMPAIGN. EVERY ROW LEAVES THE OPERATION SCHEDULE, PACKING, MULTIPLICATIVE DEPTH, AND FROZEN PLAINTEXT REFERENCE UNCHANGED.

Change	Verified effect	Status
Re-encode every plaintext per multiplication	159,744 encodings	baseline
Thread affinity, NUMA-local allocation	host placement changed	retained
Encode once, clone per use	removes about 90% of encodings	retained
Parallel batched encoding	equal to serial encoding	retained
Encode at use level, flush between stages	peak 47.5 GiB RSS	enabling
Net	checked operation schedule unchanged	

Stage flushing bounds host memory

Standalone-block samples; target-process resident set.



Stage peaks are sampled, not allocation-by-allocation maxima.

Fig. 4. Host-memory bounding through level-specific encoding and stage-local template lifetimes. The measured resident set falls after projection templates are flushed; the MLP determines the retained complete-block peak.

The four packed activation copies currently carry duplicates through dense transforms. An analyzed layout applies distinct transforms to them, combining query/key/value projections and MLP expansion chunks. At fixed model arithmetic, the projected schedule reduces dense products from 156 to 52 and dense ciphertext–plaintext multiplications from 159,744 to 53,248, lowering the total ciphertext–plaintext count by roughly 60% before mask and copy-repair overhead. An exact reconstruction proof is required before implementation.

Softmax accounts for 99.5% of boundary values, with independent cryptographic work per tile. Concurrent boundaries were unsafe under the shared crypto context, requiring sequential execution; a thread-safe context would permit concurrency.

VII. RESULTS

We evaluate the client-assisted circuit along four axes: plaintext model fidelity, encrypted numerical agreement, executed work, and resource cost. The strongest result is one complete classifier execution for a held-out genomic-signal input at the task’s full prompt length.

A. Plaintext model fidelity

Table IV and Fig. 5 show that the target carries useful signal across classification and regression tasks. On the complete polyadenylation-signal test set of 22,604 examples, the released classifier reaches 0.9124 accuracy and 0.916 F1, against the

Plaintext model reproduction

Filled: this work; open: published reference.

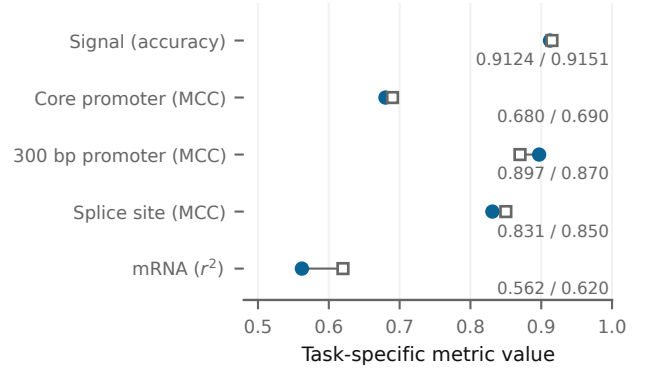


Fig. 5. Plaintext model fidelity against published references. The signal-recognition comparison uses different test splits; promoter and splice-site comparisons use the stated benchmark splits. Abundance regression is outside the encrypted scope.

0.9151 accuracy reported for this model class on a held-out quarter of that set [4]. The locally fine-tuned understanding-benchmark heads reach MCC 0.680, 0.897, and 0.831 on core-promoter, promoter, and splice-site detection, against the 0.69, 0.87, and 0.85 reported on the same splits [10]. The released abundance-regression head reaches $r^2 = 0.562$ and Pearson $r = 0.753$, against the 0.62 result reported over the Xpresso regression baseline [4], [9]. These results establish model fidelity; encrypted evaluation is still required to establish cryptographic feasibility.

Two dataset qualifications belong with these baseline results. DNAGPT does not release a head for the understanding benchmark, so we trained a linear head and evaluated the three human promoter and splice-site datasets selected for this study. For abundance regression, the historical source was recovered and its split procedure reproduced, but the resulting test set may not be gene-for-gene identical to the historical split. The abundance result is therefore model-fidelity evidence and is not used as an encrypted target.

B. Complete-classifier correctness

Correctness rests on the two-stage chain in Section IV-E. Before encryption, the independent NumPy reference is asserted against the upstream PyTorch computation at tolerance $2 \times$

TABLE IV

RELEASED DNAGPT EVALUATED LOCALLY ON THREE GENOMIC TASK FAMILIES. PUBLISHED COMPARATORS ARE DNAGPT-M FOR SIGNAL RECOGNITION AND ABUNDANCE REGRESSION, AND DNABERT-2 FOR THE THREE UNDERSTANDING-BENCHMARK TASKS. SIGNAL RECOGNITION AND ABUNDANCE REGRESSION USE RELEASED FINE-TUNED HEADS; THE UNDERSTANDING-BENCHMARK ROWS USE A LOCALLY FINE-TUNED HEAD, AS NO HEAD WAS RELEASED.

Task	Metric	This work	Reference	Source
Polyadenylation-signal recognition [‡]	accuracy	0.9124	0.9151	[4]
	F1	0.916	—	
Core promoter detection	MCC	0.680	0.69	[10]
300 bp promoter detection	MCC	0.897	0.87	[10]
Splice-site classification	MCC	0.831	0.85	[10]
mRNA abundance regression [†]	r^2	0.562	0.62	[4]
	Pearson r	0.753	—	

[‡]The reference is DNAGPT’s own reported accuracy for this model class, measured on a held-out quarter of the set; this work evaluates all 22,604 examples. The comparison is same-model but not same-split.

[†]Model-fidelity evidence only. This task is outside the encrypted scope; see Section VIII.

10^{-5} for every transformer block and the classifier head. The encrypted circuit is then compared with that validated reference under the predeclared relative infinity-norm tolerance of 4×10^{-2} .

All 12 released transformer blocks, 11 full-hidden-state client refreshes, and the released task head complete for the 103-token input. Across the block outputs, global relative error is at most 2.07×10^{-8} , and worst-token relative error is at most 1.54×10^{-7} . The final encrypted-head margin has relative error 8.56×10^{-9} and yields the same label as the frozen plaintext reference. Every block output stays within the predeclared tolerance after its refresh. This is an encrypted prediction for one held-out input, not an encrypted estimate of task accuracy over the test set.

The complete result subsumes the earlier composition checks summarized in Table V. A separate two-block execution at two tokens had established that a full-hidden-state refresh starts the next block at level 0 without increasing GPU memory. The task-length execution now verifies that mechanism across every released block and the task head.

Figure 6 shows the blockwise timing and numerical agreement within this execution.

C. Executed work

Table VI reports the checked operation schedule. Every block retains the same schedule used throughout the systems campaign. The complete run adds the released task head and asserts its own totals, so the final label cannot be attributed to silently omitting a model stage.

For one block, the 857 physical boundary crossings carry 129,162 logical values. Softmax accounts for 128,544, or 99.5%, of them; LayerNorm and GELU account for the remainder. Eight-token packing reduces the serial-equivalent dense-product count from 1,236 to 156, a $7.92\times$ structural reduction. Rotations, masking, client work, and host preparation remain, so that factor is an operation-count reduction rather than a latency speedup.

D. Measured elapsed time and work split

The complete classifier takes 6,683 s (1.86 h) from process start through completion; encrypted evaluation accounts for

Measured trajectory across the complete run

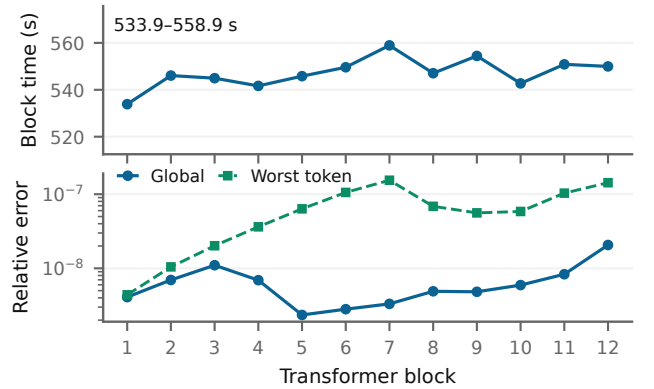


Fig. 6. Blockwise time and numerical agreement in the measured complete-classifier execution. The consecutive blocks belong to one input and one run; their variation is not independent-run variance.

6,594.75 s. The 12 blocks consume 6,565.99 s, the encrypted head 9.05 s, and the refreshes 19.71 s. The block times within this execution range from 533.9 to 558.9 s, with a mean of 547.2 s. This within-run range is about 4.5% of the maximum; it describes block consistency in one execution, not independent-run variance.

Server-side encrypted linear algebra accounts for 5,638.56 s, or 85.5% of encrypted evaluation. The in-process client boundaries account for 956.19 s, or 14.5%, and require no GPU. The measured execution is therefore server-dominated, while the data owner’s work remains CPU-only. The client fraction should decrease as $O(1/D)$ with hidden width D at fixed sequence structure because boundary values scale with D and server dense transforms with D^2 ; this is a scaling argument, not a measurement of a wider model.

Figure 7 decomposes this same measured interval two ways: by phase (process wall against blocks, refreshes, and the task head) and by responsible party (server against client).

TABLE V
ENCRYPTED EVALUATION AT THE REPORTED SCOPES. THE COMPLETE-CLASSIFIER ROW REPRESENTS ONE HELD-OUT GENOMIC-SIGNAL INPUT, NOT A TEST-SET ACCURACY MEASUREMENT.

Scope	Status	Numerical agreement	Wall time	Process peak memory
Two blocks, 2 tokens, client refresh	measured	8.48×10^{-11} after block 2	—	no growth in block 2
One block, 103 tokens	measured	4.64×10^{-9}	—	9.6 GiB GPU
Twelve blocks and task head, 103 tokens	measured	label match; 8.56×10^{-9} head margin	6,683 s	9,839 MiB GPU; 49.7 GiB host
Encrypted token-index lookup	outside evaluated boundary	—	—	—

TABLE VI
CHECKED WORK FOR ONE TASK-LENGTH BLOCK AND FOR THE MEASURED COMPLETE CLASSIFIER. PHYSICAL CLIENT CROSSINGS ARE IN-PROCESS CRYPTOGRAPHIC OPERATIONS, NOT NETWORK MESSAGES.

Operation	One block	Complete classifier	Executor
Dense encrypted products	156	1,872 blocks + 1 head	compute provider
Ciphertext–plaintext multiplications	177,734	2,133,839	compute provider
Ciphertext–ciphertext multiplications	1,506	18,076	compute provider
Rotations	8,173	—	compute provider
Slot reductions	8,776	—	compute provider
Physical client boundary crossings	857	10,299	data owner
Logical nonlinear values	129,162	1,551,081	data owner

Where the measured time goes

The same measured interval, viewed by phase and by party.

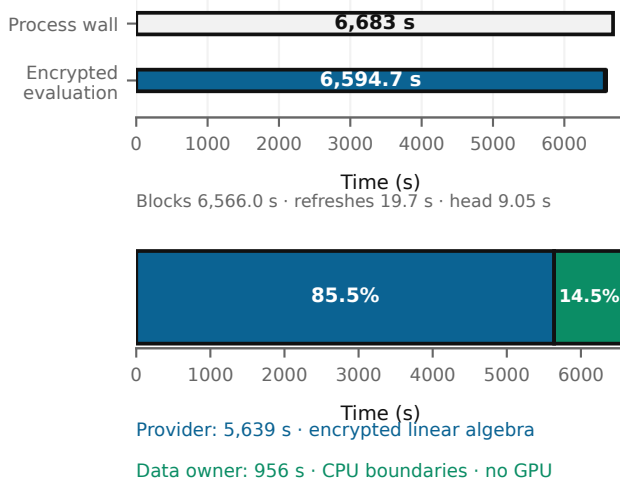


Fig. 7. Measured complete-run time, decomposed by phase and by party. Top: process wall against instrumented transformer, refresh, and task-head subtotals; the remaining interval falls outside those subtotals. Bottom: the same encrypted-evaluation interval split between the compute provider’s encrypted linear algebra and the data owner’s in-process client boundaries, which run on CPU and exclude network transport.

E. Memory

The complete process peaks at 9,839 MiB of GPU memory and 49.7 GiB of host resident memory. The GPU value is the evaluating process footprint, not device-wide occupancy. At

one-block scope, host memory follows the encoded-template lifetime: query/key/value projection reaches 24.9 GiB, attention begins after a cache flush at 12.1 GiB, and the MLP reaches 47.5 GiB while holding the largest live weight set.

The non-interactive composition configuration and the host cache expose different memory mechanisms. The former exceeded its tested 80 GB GPU envelope while loading a deeper context and evaluation keys. Client re-encryption bounds multiplicative depth between declared boundaries and thereby moves that accelerator barrier. Separately, stage-bounded eviction prevents encoded host templates from accumulating; an unbounded cache was terminated at 60.4 GiB before the retained policy reduced the block peak to 47.5 GiB.

F. Sequence-length boundary

Genomic-signal recognition uses 600 base pairs, which become 100 sequence tokens under 6-mer tokenization; three task-format tokens produce the 103-token measured prompt. Eight-token packing maps it to 13 ciphertext groups. Core-promoter, promoter, and splice-site prompts use fewer groups under the same layout, but their complete encrypted classifiers have not been measured; any elapsed-time estimate for them is a fixed-circuit projection.

The abundance-regression task lies outside the encrypted scope. Its 10,500 base pairs produce 1,755 tokens and 220 ciphertext groups. The causal schedule would require 24,310 score tiles, placing it in a different circuit-size regime from the classification prompts.

Figure 8 scales the measured complete-program time by ciphertext-group count for an illustrative comparison. The alternative task classifiers and their heads have not been measured; the projections do not separately model quadratic attention cost or fixed overhead.

Complete run and shorter-task projections

Solid: measured; hatched: group-linear wall-time model.

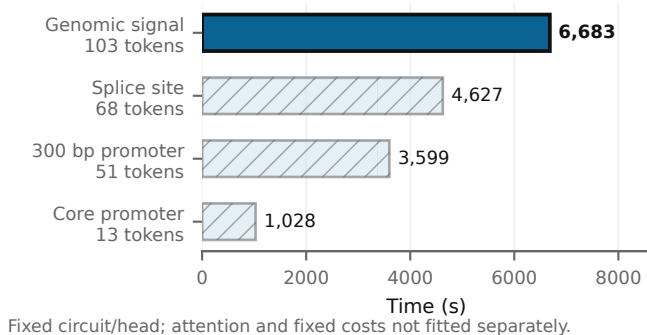


Fig. 8. Sequence-length comparison. The genomic-signal classifier is the measured complete run; hatched values assume whole-program time proportional to ciphertext-group count. This illustrative model holds the circuit and head fixed and does not separately model quadratic attention or fixed overhead.

G. Feasibility

Arithmetic feasibility is established for one complete released-weight genomic-signal classifier at its full prompt length. Every transformer block, every declared refresh, and the released head complete within the tested memory envelope; the final label matches the frozen plaintext reference; and intermediate errors remain far inside the acceptance tolerance. This verdict applies from encrypted embedded vectors through the released head; it does not cover private token-index lookup or prediction agreement over the full genomic-signal test set.

H. Practicality, judged separately

The measured cost is substantial: one encrypted example takes 1.86 h on one A100-class GPU with CPU-only client assistance. That value is a real latency measurement for the evaluated program, not a projection. It is also one execution with in-process boundaries, without a network or an independent repetition. The evidence therefore characterizes the present cost but does not establish variance, throughput, transport overhead, or suitability for a particular clinical or research service. Practicality remains a deployment-specific question rather than a consequence inferred from the affirmative feasibility result.

VIII. LIMITATIONS

The strongest measurement is one complete execution of all 12 released transformer blocks and the genomic-signal task head for one held-out input at the full 103-token prompt length. It establishes numerical agreement for that classifier execution, including the final prediction, but does not estimate encrypted task accuracy over the full test set. The complete run has one clean sample and has not been independently repeated to quantify run-to-run variation.

Client and server are roles inside one program on one node. Boundary calls perform cryptographic operations directly, without ciphertext serialization, network transport, or a key-custody service. A physical client boundary crossing is therefore not

a network message. A deployment experiment must measure payload volume, dependency phases, bandwidth sensitivity, transport latency, and production key handling rather than infer them from the in-process counter.

Encryption begins at embedded numeric vectors. Tokenization, token-index lookup in the embedding table, and construction of the initial embedding occur before the encrypted boundary. Private lookup therefore remains unresolved even though the classifier computation after embedding now completes.

Model confidentiality remains operational rather than cryptographic. Because the data owner sees full activations at every nonlinearity, the transcript divides the network into shallow segments with known nonlinearities. Weight recovery becomes inexpensive per-segment regression instead of a global inversion problem. Server-side weight placement can discourage casual copying but cannot protect the model from a determined client; the deployment keeps the model on the server because the server operates it.

CKKS decryption is approximate. The client never returns a decoded value to the server, only a freshly encrypted result, but the server selects the ciphertext supplied to each declared boundary. The semi-honest threat model rules out deviations from that schedule. The current re-encryption path applies no noise flooding. The measured numerical headroom does not establish a secure flooding parameter, which remains future work. The adversary exclusions are defined in Section IV; the results do not extend beyond them.

IX. CONCLUSION

Client-assisted CKKS executes all 12 released DNAGPT transformer blocks and the genomic-signal task head for one held-out 103-token input. Intermediate errors remain within 1.54×10^{-7} , the encrypted head margin has 8.56×10^{-9} relative error, and the final label matches the independently validated plaintext reference. The complete execution peaks at 9,839 MiB of process GPU memory. Arithmetic feasibility is therefore affirmative from encrypted embedded vectors through the released head.

The measured cost is 6,683 s for one example on a single accelerator. Server-side encrypted linear algebra accounts for 85.5% of encrypted evaluation and CPU-only client boundaries for 14.5%. These measurements characterize present cost without establishing deployment practicality: independent-run variation and network transport remain unmeasured, and private token-index lookup remains outside the encrypted boundary.

The comparison with non-interactive CKKS identifies architecture, rather than numerical accuracy, as the decisive constraint. Polynomial nonlinearities completed one real-weight block, but their composition configuration exceeded the tested 80 GB GPU-memory envelope. Exact nonlinearities and fresh encryption at declared client boundaries bound the depth of each encrypted segment. Separately, encoding public weight diagonals once and evicting their templates at stage boundaries bounds host memory without changing the checked operation schedule.

Future work

The immediate measurement priority is an independent repetition of the complete execution, followed by a serialized client/server transport experiment. Systems work should cache fixed mask plaintexts and evaluate distinct transforms across packed activation copies. A thread-safe cryptographic context could expose independent client boundaries to concurrency. Protocol closure also requires a validated noise-flooding budget for re-encryption and private token-index lookup.

ETHICS STATEMENT

This study involved no human participants, specimens, or private clinical records. It used public reference datasets with recorded provenance. The work makes no clinical-performance claim and does not present the prototype as a deployable privacy guarantee beyond its stated threat model.

DATA AVAILABILITY STATEMENT

All evaluation datasets are public. Their original sources, versions, preprocessing, and any recovery route used are recorded in the repository's data-provenance document. Sequence data, model weights, and cryptographic keys are not redistributed.

CODE AVAILABILITY STATEMENT

The evaluation harnesses, encrypted-evaluation implementation, environment definitions, and the evidence record for the paper are maintained at https://github.com/Georgakopoulos-Soares-lab/private_genomic_dnagpt. No repository-level reuse license has yet been assigned.

REFERENCES

- [1] M. Gymrek, A. L. McGuire, D. Golan, E. Halperin, and Y. Erlich, "Identifying personal genomes by surname inference," *Science*, vol. 339, no. 6117, pp. 321–324, 2013.
- [2] Y. Erlich, T. Shor, I. Pe'er, and S. Carmi, "Identity inference of genomic data using long-range familial searches," *Science*, vol. 362, no. 6415, pp. 690–694, 2018.
- [3] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing directive 95/46/EC (general data protection regulation)," Official Journal of the European Union, L 119, pp. 1–88, May 2016. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [4] D. Zhang, W. Zhang, Y. Zhao, J. Zhang, B. He, C. Qin, and J. Yao, "DNAGPT: A generalized pre-trained tool for versatile DNA sequence analysis tasks," *arXiv preprint arXiv:2307.05628*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.05628>
- [5] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology – ASIACRYPT 2017*, ser. Lecture Notes in Computer Science, vol. 10624, 2017, pp. 409–437.
- [6] J. Moon, D. Yoo, X. Jiang, and M. Kim, "THOR: Secure transformer inference with homomorphic encryption," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2025, pp. 3765–3779.
- [7] S. Biswas, P. Chartier, A. Dhasade, T. Jurien, D. Kerriou, A.-M. Kermarec, M. Lemou, F. Tranie, M. de Vos, and M. Vujanovic, "Practical and private hybrid ML inference with fully homomorphic encryption," *arXiv preprint arXiv:2509.01253*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.01253>
- [8] M. Kalkatawi, A. Magana-Mora, B. Jankovic, and V. B. Bajic, "DeepGSR: an optimized deep-learning structure for the recognition of genomic signals and regions," *Bioinformatics*, vol. 35, no. 7, pp. 1125–1132, 2019.
- [9] V. Agarwal and J. Shendure, "Predicting mRNA abundance directly from genomic sequence using deep convolutional neural networks," *Cell Reports*, vol. 31, no. 7, p. 107663, 2020.
- [10] Z. Zhou, Y. Ji, W. Li, P. Dutta, R. V. Davuluri, and H. Liu, "DNABERT-2: Efficient foundation model and benchmark for multi-species genomes," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=MLQB4EZE1>
- [11] A. Al Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, I. Quah, Y. Polyakov, Saraswathy R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca, "OpenFHE: Open-source fully homomorphic encryption library," in *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. Association for Computing Machinery, 2022, pp. 53–63.
- [12] Zama, "Concrete ML: a privacy-preserving machine learning library using fully homomorphic encryption for data scientists," 2022, accessed: Sep. 7, 2026. [Online]. Available: <https://github.com/zama-ai/concrete-ml>
- [13] —, "TFHE-rs: A pure Rust implementation of the TFHE scheme for boolean and integer arithmetics over encrypted data," 2022, accessed: Sep. 7, 2026. [Online]. Available: <https://github.com/zama-ai/tfhe-rs>
- [14] —, "Inference," Concrete ML documentation, accessed: Sep. 7, 2026. [Online]. Available: <https://docs.zama.org/concrete-ml/llms/inference>
- [15] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "GAZELLE: A low latency framework for secure neural network inference," in *27th USENIX Security Symposium*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 1651–1669. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar>
- [16] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, "Iron: Private inference on transformers," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 15 718–15 731.
- [17] Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider, "BOLT: Privacy-preserving, accurate and efficient inference for transformers," in *2024 IEEE Symposium on Security and Privacy*. IEEE, 2024, pp. 4753–4771.
- [18] Z. Li, K. Yang, J. Tan, W.-j. Lu, H. Wu, X. Wang, Y. Yu, D. Zhao, Y. Zheng, M. Guo, and J. Leng, "Nimbus: Secure and efficient two-party inference for transformers," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 21 572–21 600.
- [19] W.-j. Lu, Z. Huang, Z. Gu, J. Li, J. Liu, C. Hong, K. Ren, T. Wei, and W. Chen, "BumbleBee: Secure two-party inference framework for large transformers," in *32nd Annual Network and Distributed System Security Symposium*. The Internet Society, 2025.
- [20] T.-T. Kuo, X. Jiang, H. Tang, X. Wang, T. Bath, D. Bu, L. Wang, A. Harmanci, S. Zhang, D. Zhi, H. J. Sofia, and L. Ohno-Machado, "iDASH secure genome analysis competition 2018: blockchain genomic data access logging, homomorphic encryption on GWAS, and DNA segment searching," *BMC Medical Genomics*, vol. 13, no. Suppl 7, p. 98, 2020.
- [21] G. S. Çetin, H. Chen, K. Laine, K. Lauter, P. Rindal, and Y. Xia, "Private queries on encrypted genomic data," *BMC Medical Genomics*, vol. 10, no. Suppl 2, p. 45, 2017.
- [22] C. Agulló-Domingo, O. Vera-López, S. Guzelhan, L. Daksha, A. El Jerari, K. Shivdikar, R. Agrawal, D. Kaeli, A. Joshi, and J. L. Abellán, "FIDESlib: A fully-fledged open-source FHE library for efficient CKKS on GPUs," *arXiv preprint arXiv:2507.04775*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.04775>
- [23] Z. Gong, R. Ran, F. Yao, and W. Wen, "Scaling long-sequence homomorphic encrypted transformer inference via hybrid parallelism on multi-GPU systems," in *Proceedings of the 40th ACM International Conference on Supercomputing*. Association for Computing Machinery, 2026, pp. 1206–1219.
- [24] Y. Bae, J. H. Cheon, G. Hanrot, J. H. Park, and D. Stehlé, "Fast homomorphic linear algebra with BLAS," *Journal of Cryptology*, vol. 39, no. 3, p. 25, 2026.